

ASIX PILDORAS:

DOCKER Y DOCKERHUB:

COMPROMETIENDO LA SEGURIDAD

DE DOCKER:

Parte Práctica



INDICE

Introducción.....	3
Que es un Proof of Concept?.....	3
Demostración Práctica 1 : Proof Of Concept.....	3
Pasos que hemos seguido para nuestro Ataque de Cadena:.....	3
Explicación: Proof Of Concept.....	15
1. ¿Cómo Funcionan los Dockerfiles?.....	15
2. Creación y Distribución en Docker Hub.....	15
3. Ataque a la Cadena de Suministro.....	15
4. Formas de Ataques a la Cadena de Suministro Usando Docker.....	16
5. Prevención y Mitigación.....	16
Demostración Práctica 2 : Ejemplo Complejo.....	16
Pasos que hemos seguido para nuestro Ataque de Cadena:.....	16
Principales vulnerabilidades a ejecutar.....	17
Explicación: Ejemplo Complejo.....	21
1. ¿Cómo Funcionan los Dockerfiles?.....	21
2. Generación del nginx.conf Vulnerable.....	22
3. Build de la Imagen Maliciosa.....	22
4. Validación de la Imagen.....	22
5. Autenticación con Docker Hub.....	23
6. Tag y Push de la Imagen.....	23
7. Verificación desde Docker Hub.....	23
Conclusión.....	23
Apendice:.....	24

Integrantes:

Bryan Canton

Nikolas Carmona

Alexandru Gabriel Babiac

Mukul Kumar Dadar

Alejandro Upegui Carvajal

Nadia Cayo Moya

Erec Dionisio Pereira

Muhammad Haris Ahmed Fatima

Guiu Ferrer Garcia

Frank Garcia Nebot

Hugo Castella

Alejandro Martinez

Introducción

En nuestro caso, dividiremos la práctica en dos partes: primero realizaremos un **Proof of Concept**, en el que implementaremos una versión sencilla que nos permita validar los aspectos básicos del proyecto (Path Hijacking, Vulnerabilidades de Docker). Posteriormente, desarrollaremos un ejemplo más complicado que represente un escenario más realista y avanzado, aplicando los conocimientos adquiridos y mejorando la complejidad del sistema.

Que es un Proof of Concept?

Un *Proof of Concept* (PoC), o **Prueba de Concepto**, es una demostración preliminar cuyo objetivo es comprobar si una idea, propuesta o solución es técnicamente viable. Sirve para validar que un enfoque puede funcionar en la práctica antes de invertir mayores recursos en su desarrollo completo. En este tipo de pruebas no se busca un producto final, sino verificar que el concepto base cumple con los requisitos fundamentales y puede aplicarse de manera efectiva.

En nuestro caso, dividiremos la práctica en dos partes: primero realizaremos un **Proof of Concept**, en el que implementaremos una versión sencilla que nos permita validar los aspectos básicos del proyecto. Posteriormente, desarrollaremos un ejemplo más complicado que represente un escenario más realista y avanzado, aplicando los conocimientos adquiridos y mejorando la complejidad del sistema.

Demostracion Práctica 1 : Proof Of Concept

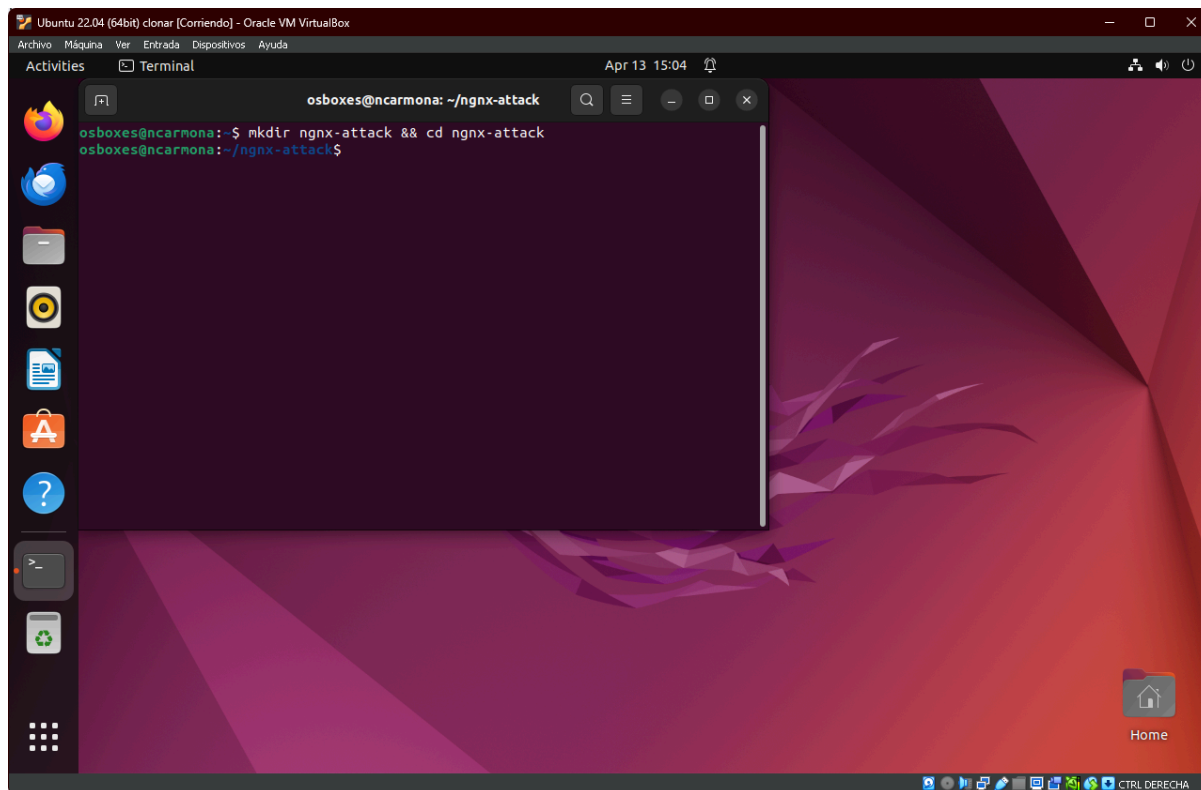
Pasos que hemos seguido para nuestro Ataque de Cadena:

- Generar un servicio de nginx en un contenedor Docker para hacer un ataque en cadena.
- Modificarlo para que ejecute el ataque en cadena, por ejemplo, cambiando un carácter para que descargue el nuestro, como un alias, y que les salte un mensaje diciendo que han sido hackeados. La descarga puede ser exactamente igual que el programa que queremos imitar, y que dé como resultado un mensaje **"¡HAS SIDO HACKEADO!"**
- Crear y subir a github, con su readme, una imagen vulnerable, además de dockerhub.
- Poner en este doc el enlace al repositorio de github

Declaramos el Dockerfile malicioso, lo basaremos en nginx

Creemos una carpeta de proyecto:

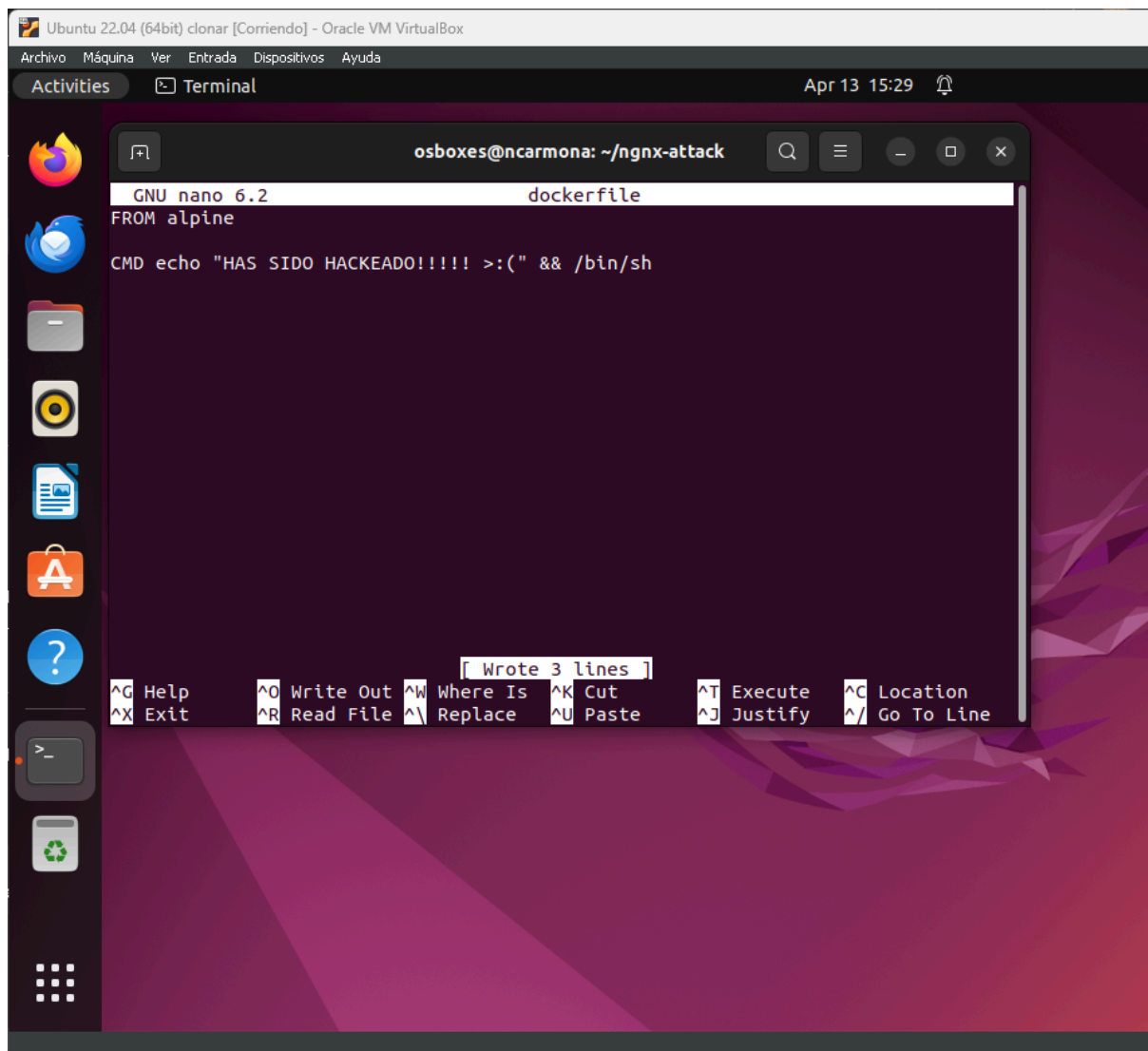
`mkdir ngx-attack && cd ngx-attack`



Creemos el Dockerfile:

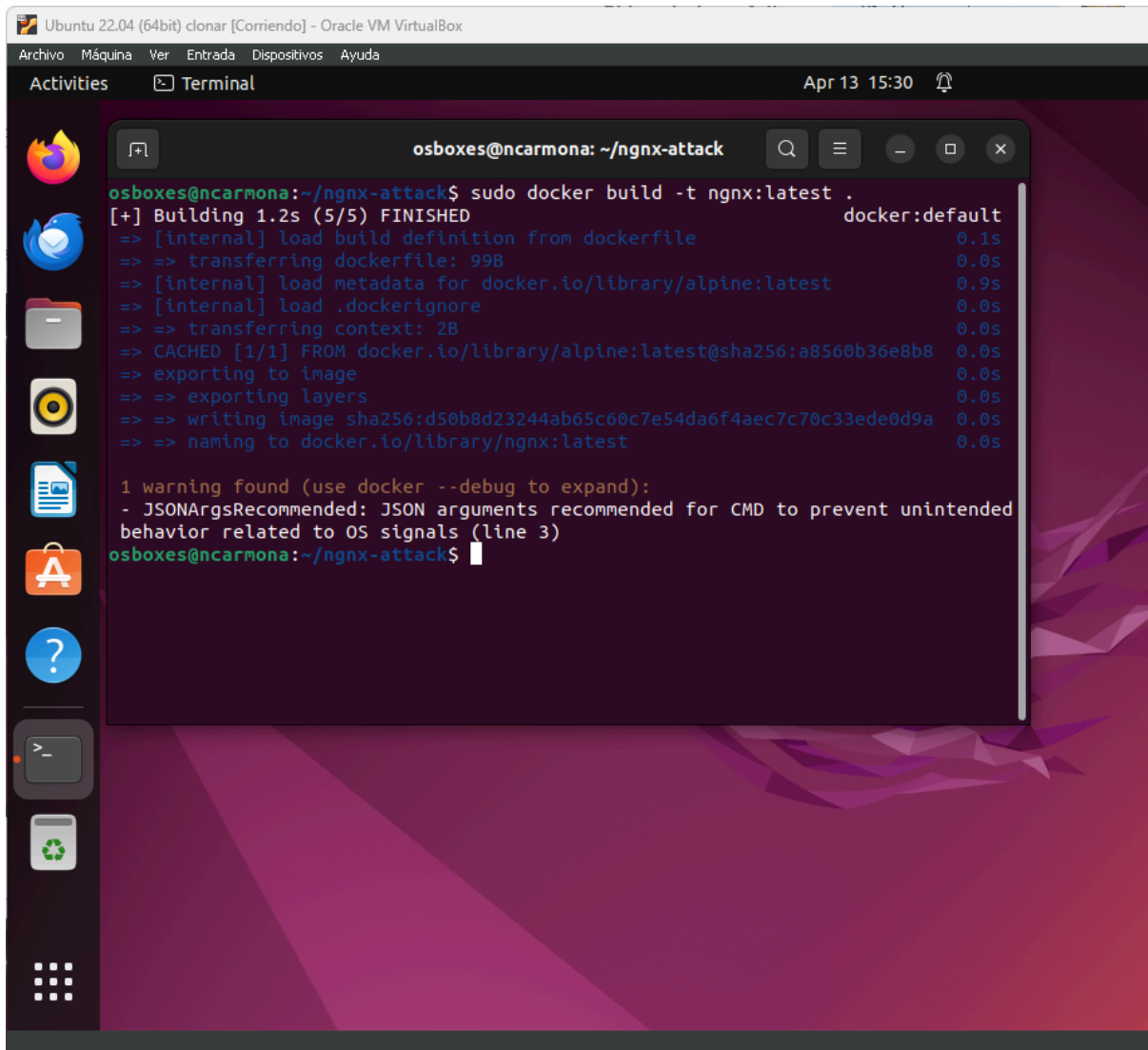
`FROM alpine`

`CMD echo "¡HAS SIDO HACKEADO!" && /bin/sh`



Buileamos el Dockerfile

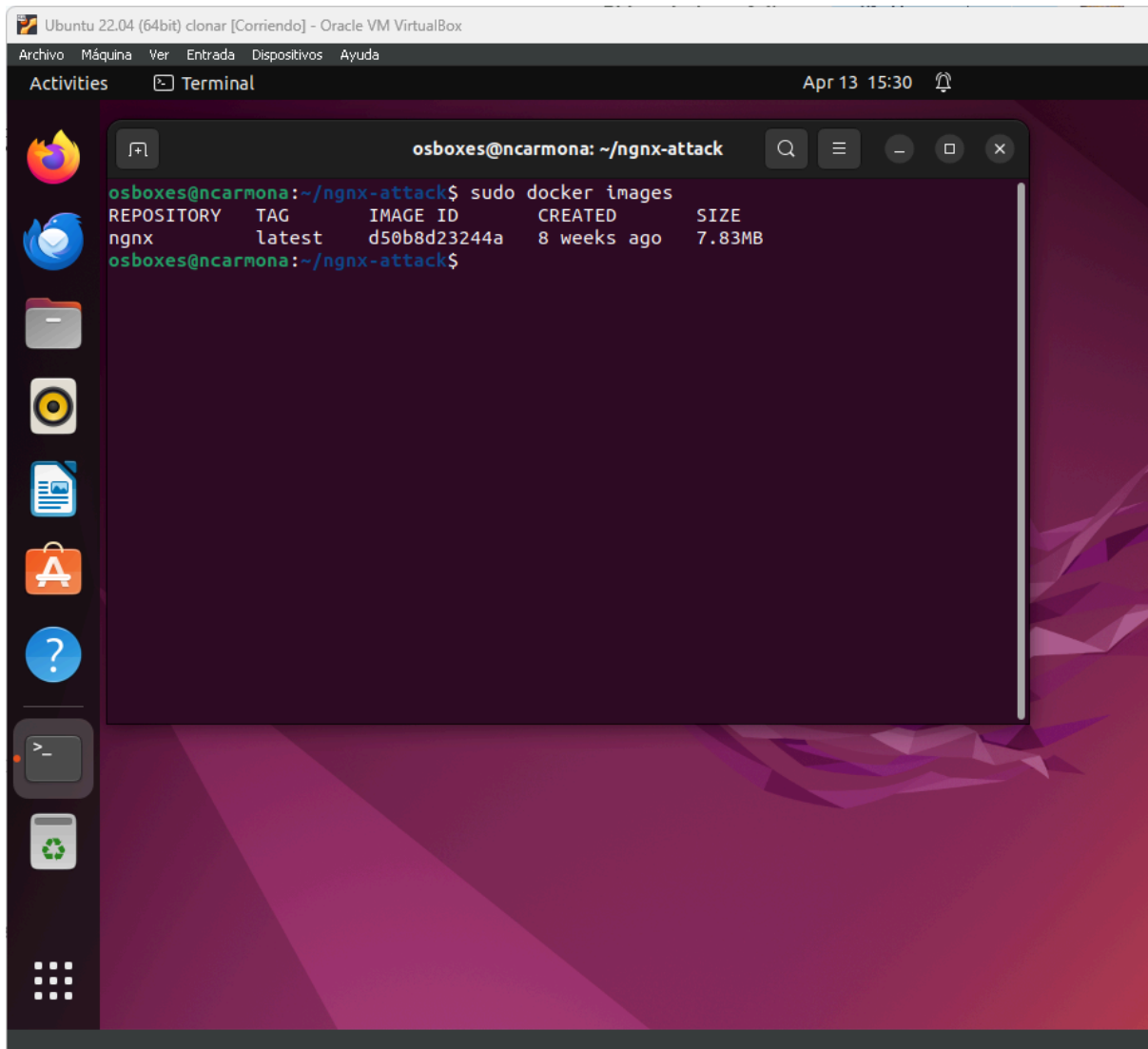
`docker build -t ngx:latest .`



```
osboxes@ncarmona: ~/ngx-attack
osboxes@ncarmona:~/ngx-attack$ sudo docker build -t ngx:latest .
[+] Building 1.2s (5/5) FINISHED
=> [internal] load build definition from dockerfile                                0.1s
=> => transferring dockerfile: 99B                                                0.0s
=> [internal] load metadata for docker.io/library/alpine:latest                  0.9s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 2B                                                    0.0s
=> CACHED [1/1] FROM docker.io/library/alpine:latest@sha256:a8560b36e8b8         0.0s
=> exporting to image                                                            0.0s
=> => exporting layers                                                            0.0s
=> => writing image sha256:d50b8d23244ab65c60c7e54da6f4aec7c70c33ede0d9a       0.0s
=> => naming to docker.io/library/ngx:latest                                    0.0s

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended
behavior related to OS signals (line 3)
osboxes@ncarmona:~/ngx-attack$
```

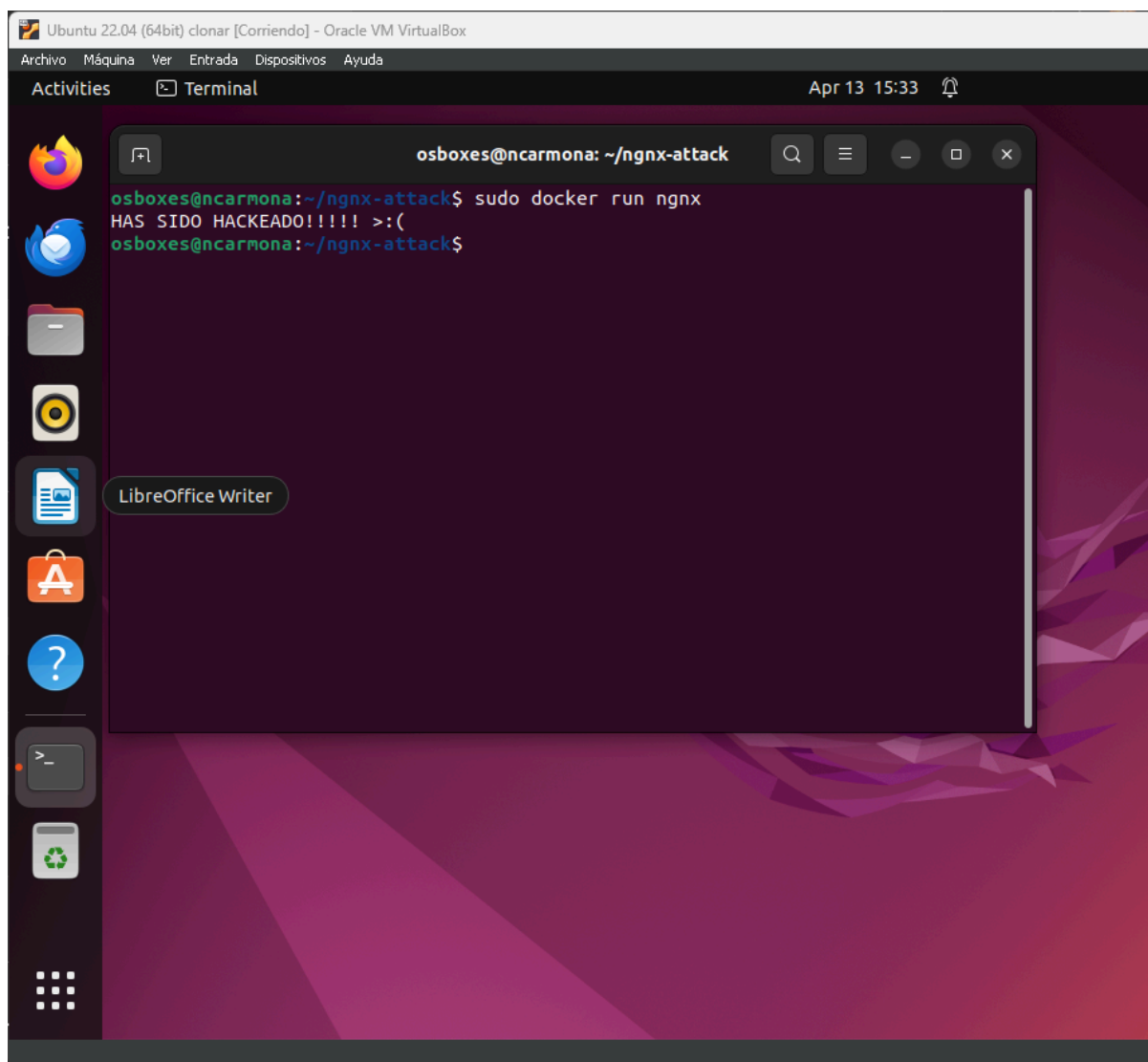
Listamos las imágenes para asegurarnos de que se ha creado
docker images



Veremos como se ha creado la imagen correctamente.

Probamos localmente la imagen para ver que se inicia correctamente

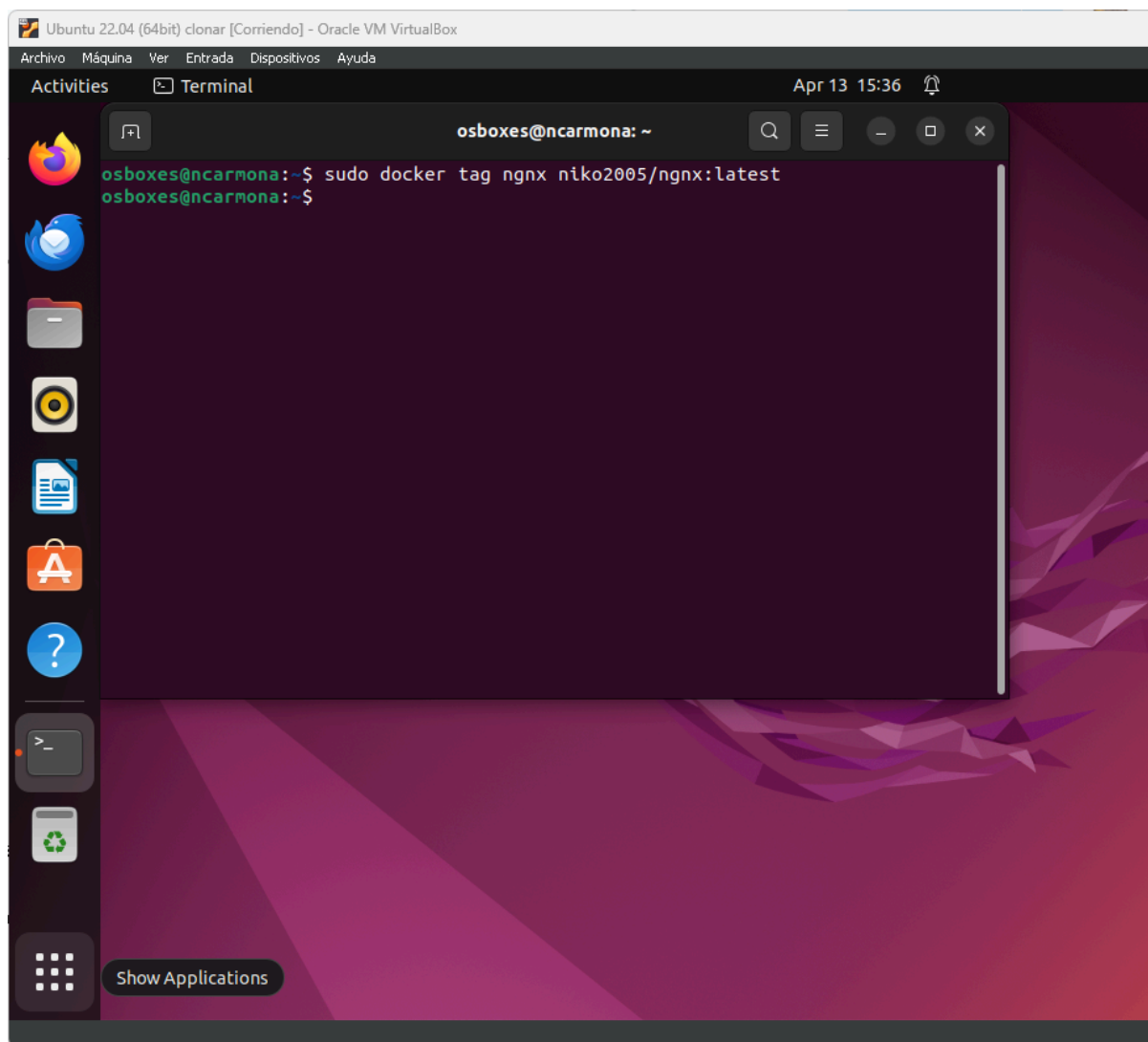
`docker run -d -p 8080:80 ngx`



Veremos el mensaje correctamente.

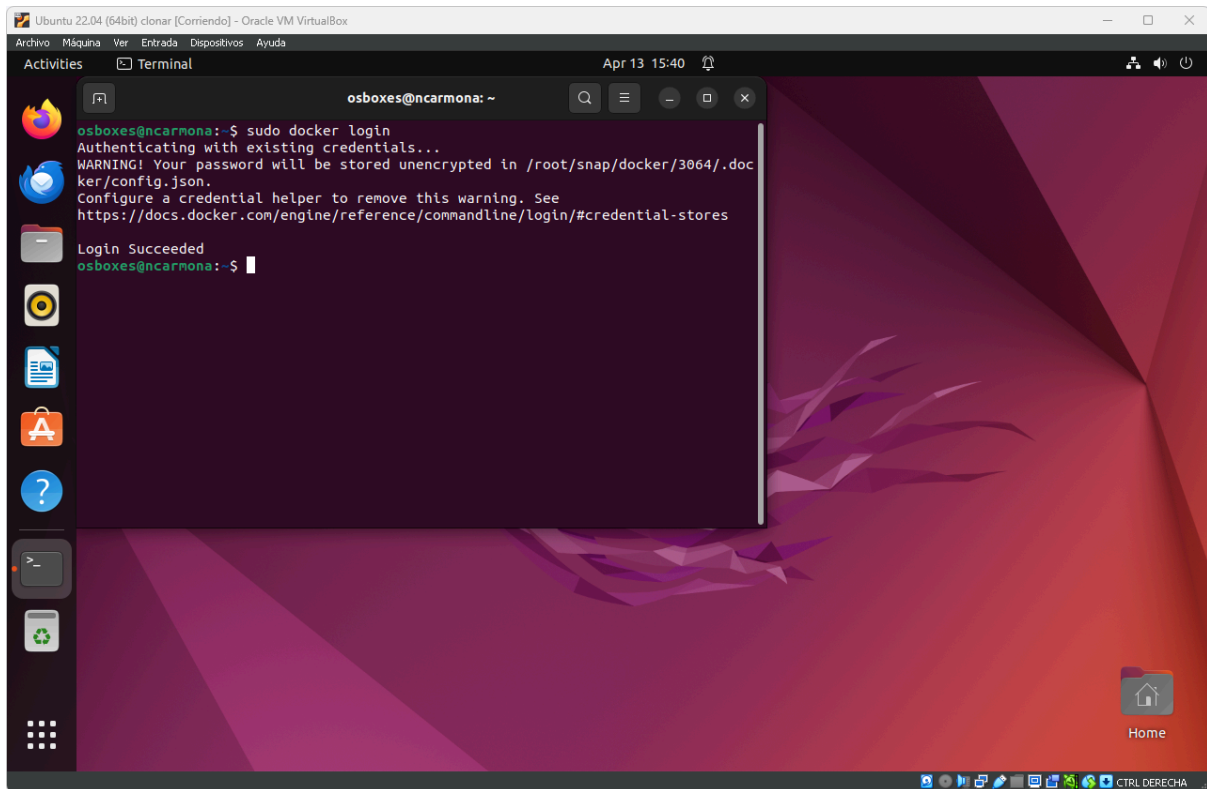
Tageamos la imagen antes de subirla

`docker tag ngx tu_usuario_dockerhub/ngx:latest`

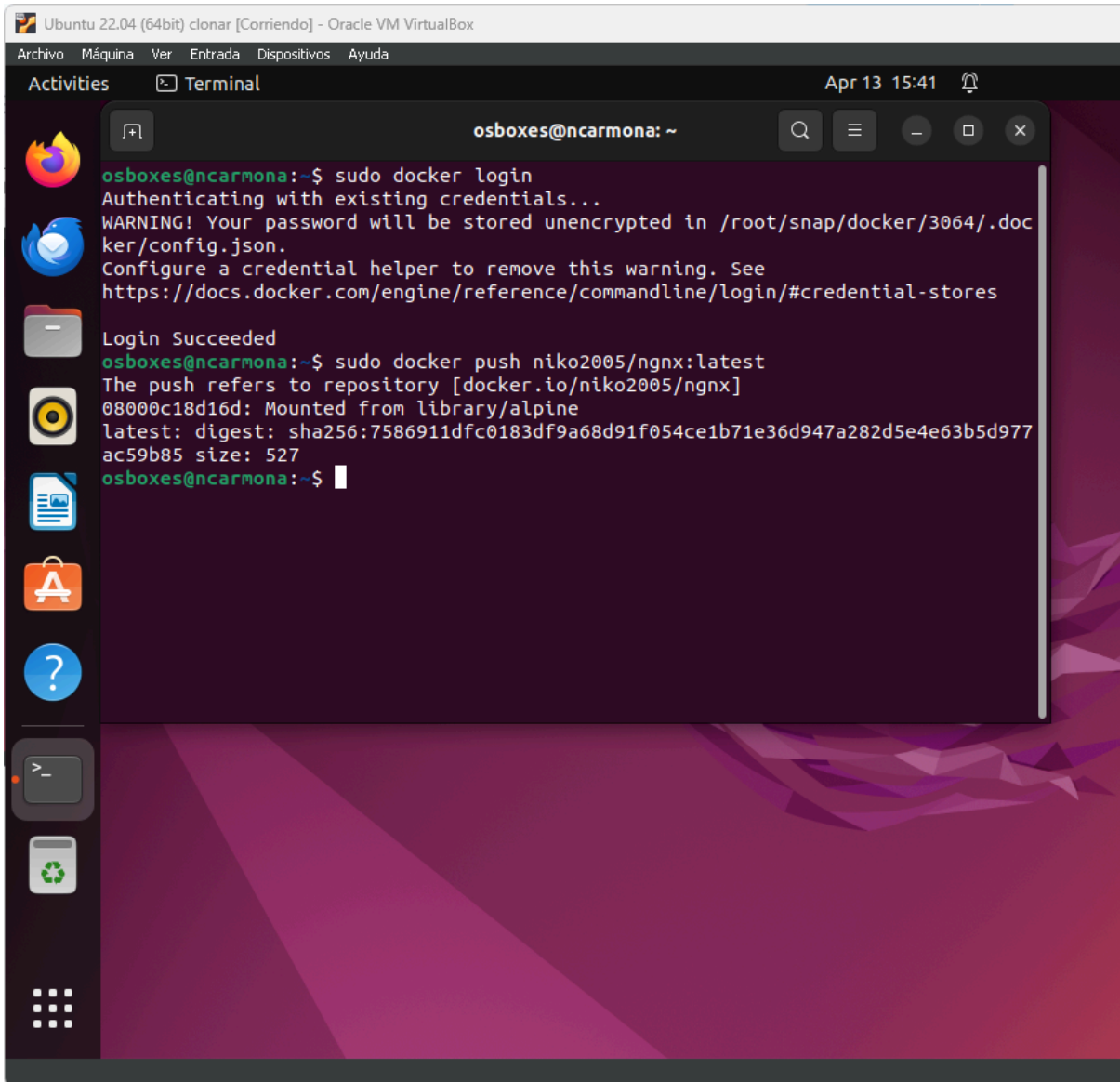


Subimos la imagen a DockerHub

`docker login`



`docker push tu_usuario_dockerhub/ngnx:latest`

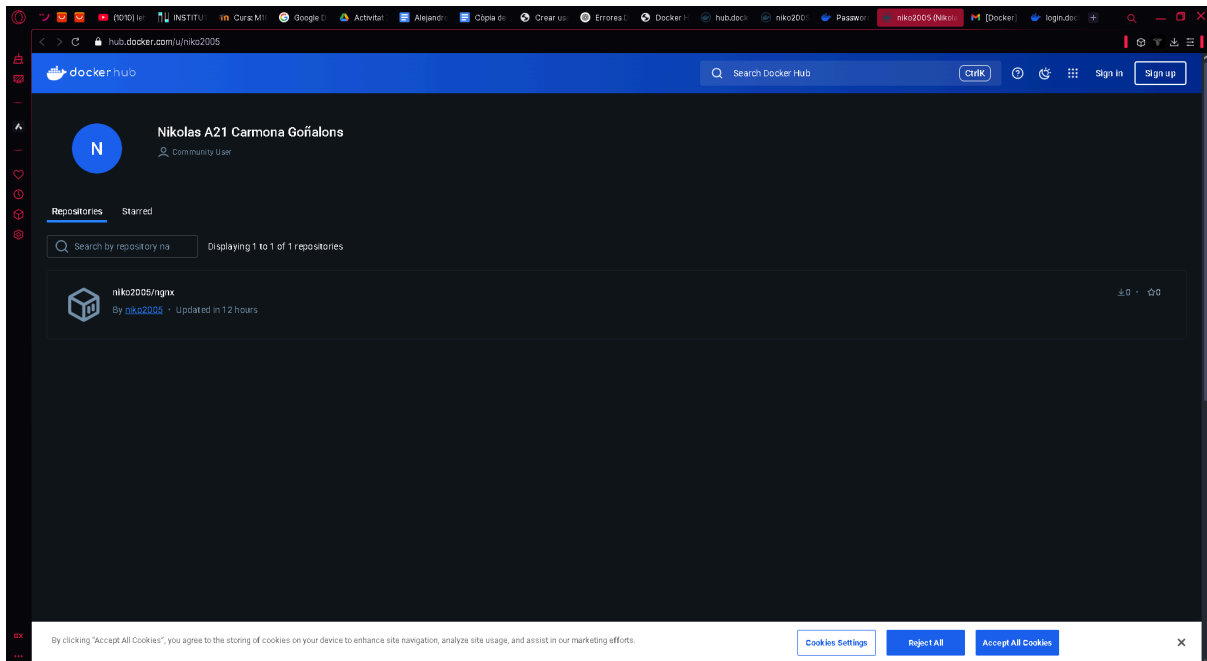


Ubuntu 22.04 (64bit) clonar [Corriendo] - Oracle VM VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

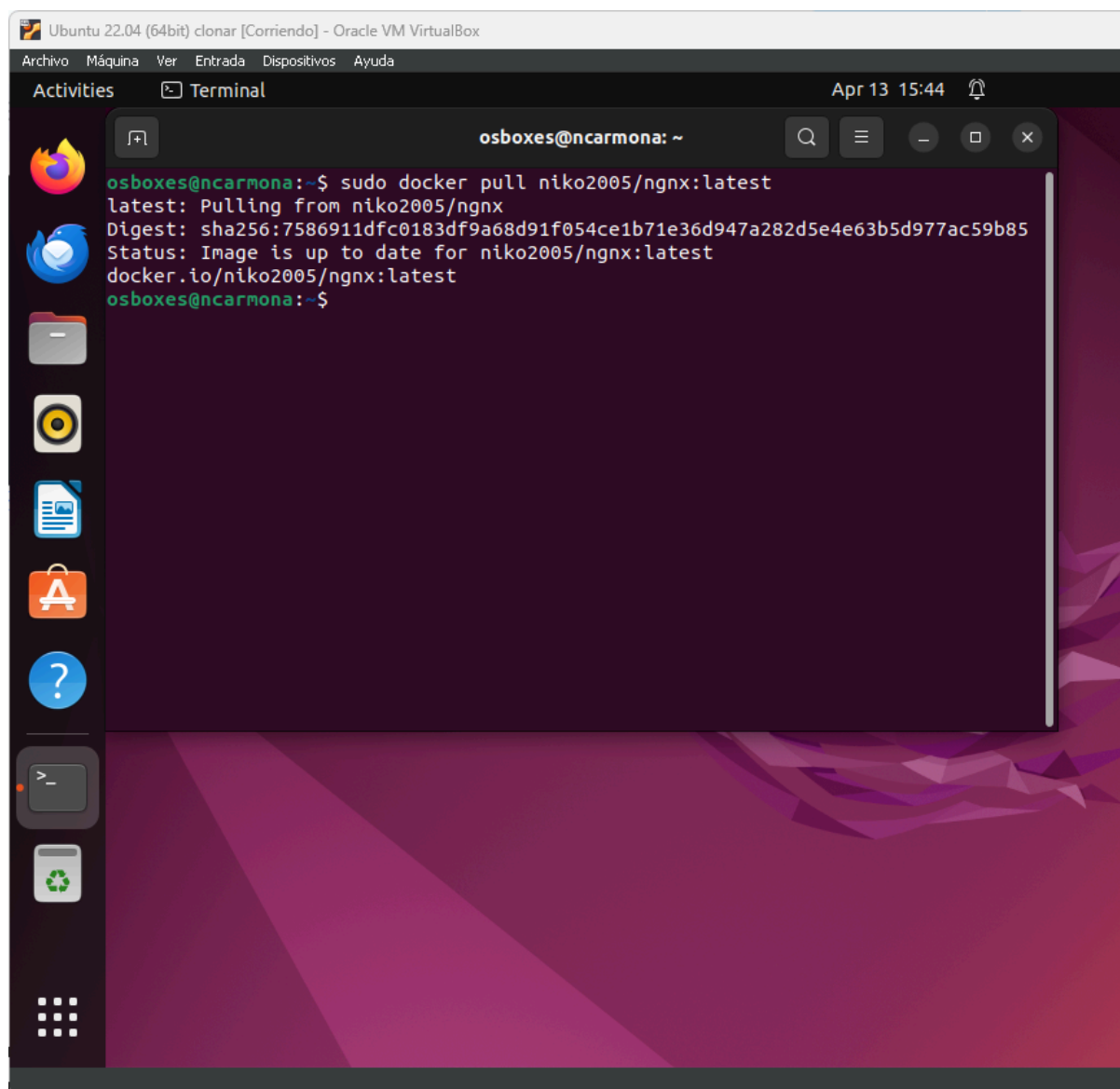
Activities Terminal Apr 13 15:41

```
osboxes@ncarmona: ~  
osboxes@ncarmona:~$ sudo docker login  
Authenticating with existing credentials...  
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credential-stores  
  
Login Succeeded  
osboxes@ncarmona:~$ sudo docker push niko2005/nginx:latest  
The push refers to repository [docker.io/niko2005/nginx]  
08000c18d16d: Mounted from library/alpine  
latest: digest: sha256:7586911dfc0183df9a68d91f054ce1b71e36d947a282d5e4e63b5d977  
ac59b85 size: 527  
osboxes@ncarmona:~$
```

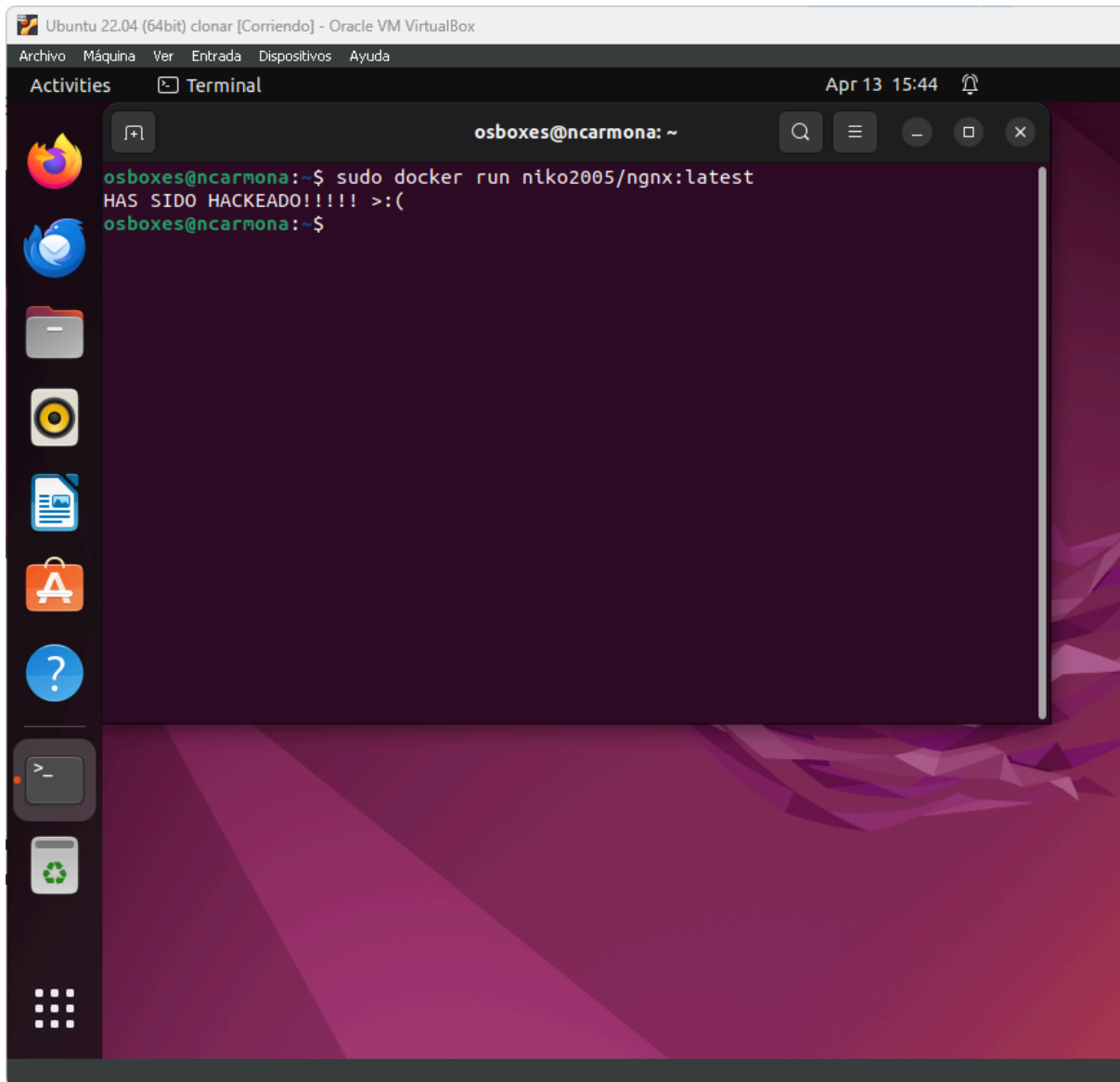


Vemos como se ha subido correctamente a nuestro usuario de DockerHub.

Nos bajamos la imagen de DockerHub para cerciorarnos de que funciona todo
`docker pull tu_usuario_dockerhub/nginx:latest`



docker run tu_usuario_dockerhub/nginx



Vemos como muestra el mensaje correctamente, que es lo que necesitabamos.

A continuacion dejamos el enlace a github, donde podemos encontrar tambien la imagen de dockerhub que hemos creado.

<https://github.com/dbrMy/P-ndola-4---Docker/tree/main>

Explicación: Proof Of Concept

1. ¿Cómo Funcionan los Dockerfiles?

Un Dockerfile es un archivo que define las instrucciones para construir una imagen de Docker. En nuestro caso, modificamos una imagen base confiable de Alpine para ejecutar un comando malicioso que muestra el mensaje "¡HAS SIDO HACKEADO!" cuando el contenedor se ejecuta. Este archivo fue sencillo:

```
FROM alpine
CMD echo "¡HAS SIDO HACKEADO!" && /bin/sh
```

Este Dockerfile crea una imagen que, al ser ejecutada, muestra el mensaje y luego abre una shell, permitiendo ejecutar comandos dentro del contenedor.

2. Creación y Distribución en Docker Hub

Después de construir la imagen, la subimos a Docker Hub, un repositorio público donde los usuarios pueden compartir imágenes. Al ser un repositorio público, cualquier usuario podría descargar y ejecutar esta imagen sin saber que contiene código malicioso.

Pasos clave:

Build: Construimos la imagen con docker build.

Push: Subimos la imagen con docker push a Docker Hub, donde otros usuarios pueden acceder a ella.

3. Ataque a la Cadena de Suministro

Este ataque es un ejemplo de un ataque a la cadena de suministro de software. Al manipular una imagen confiable (como alpine), un atacante puede distribuirla a través de Docker Hub, exponiendo a los usuarios a riesgos de seguridad. Los usuarios que descargan y ejecutan esta imagen no verifican su contenido, lo que les permite ejecutar código malicioso sin saberlo.

4. Formas de Ataques a la Cadena de Suministro Usando Docker

Los atacantes pueden explotar Docker Hub y Dockerfiles de varias maneras:

Inyección de código malicioso en imágenes populares, que se ejecuta al iniciar el contenedor.

Desbordamiento de memoria o DoS, provocando que el contenedor o los servicios asociados se caigan.

Robo de credenciales mediante la copia de archivos sensibles del sistema host al contenedor.

Escalada de privilegios, lo que permite al atacante tomar control total del sistema.

5. Prevención y Mitigación

Para prevenir estos ataques, es importante:

Verificar imágenes antes de ejecutarlas, preferiblemente usando imágenes oficiales.

Escanear vulnerabilidades con herramientas como Clair o Anchore.

Revisar Dockerfiles para asegurarse de que no contienen instrucciones maliciosas.

Limitar privilegios en los contenedores, evitando ejecutar contenedores como root.

Demostración Práctica 2 : Ejemplo Complejo

Pasos que hemos seguido para nuestro Ataque de Cadena:

- Generar un servicio de nginx en un contenedor Docker para hacer un ataque en cadena.
- Modificarlo para que ejecute el ataque en cadena, por ejemplo, cambiando un carácter para que descargue el nuestro, como un alias, y que les salte un mensaje diciendo que han sido hackeados. La

descarga puede ser exactamente igual que el programa que queremos imitar, y que dé como resultado la ejecución de varios comandos maliciosos.

- Crear y subir a github, con su readme, una imagen vulnerable, así como subirlo a DockerHub
- Poner en este doc el enlace al repositorio de github

Principales vulnerabilidades a ejecutar

- Ejecutamos todo como root (mala práctica).
- Exponemos credenciales en variables de entorno.
- Abrimos más puertos innecesarios (22 para SSH, 3306 para MySQL).
- Instalamos herramientas que podrían facilitar ataques (netcat, curl, wget).
- Configuramos Nginx para servir archivos sensibles, permitiendo acceso a /etc/passwd.
- Deshabilitamos seguridad de Nginx (sin límites de request body, sin protección contra ataques).

Declaramos el dockerfile maliciosos, lo basaremos en nginx

```
GNU nano 6.2          nginx *
FROM nginx:latest

# Copiar configuración insegura de Nginx
COPY nginx.conf /etc/nginx/nginx.conf

# Configurar permisos inseguros (dar acceso a todos)
RUN chmod -R 777 /usr/share/nginx/html

# Deshabilitar logs (mala práctica)
RUN ln -sf /dev/null /var/log/nginx/access.log
RUN ln -sf /dev/null /var/log/nginx/error.log
█

# Exponer el puerto 80
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Generamos un nginx.conf que proporcione configuraciones vulnerables al servicio nginx

```
GNU nano 6.2 nginx.conf *
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    server {
        listen 80;
        server_name _;

        # Permetir listado de directorios (exposición de archivos)
        autoindex on;
        autoindex_exact_size off;
        autoindex_format html;
        autoindex_localtime on;

        location / {
            root /usr/share/nginx/html;
            index index.html;
        }
    }
}
```

Buildemos el dockerfile

```
linux@linux-VirtualBox:~/ngx$ sudo docker build -t ngx -f ~/ngx/ .
2025/02/25 16:41:23 in: []string{}
2025/02/25 16:41:23 Parsed entitlements: []
[+] Building 8.8s (10/10) FINISHED docker:default
=> [internal] load build definition from ngx 0.1s
=> => transferring dockerfile: 427B 0.0s
=> [internal] load metadata for docker.io/library/nginx:l 7.6s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 32B 0.0s
=> [1/5] FROM docker.io/library/nginx:latest@sha256:9d6b5 0.0s
=> CACHED [2/5] COPY nginx.conf /etc/nginx/nginx.conf 0.0s
=> CACHED [3/5] RUN chmod -R 777 /usr/share/nginx/html 0.0s
=> CACHED [4/5] RUN ln -sf /dev/null /var/log/nginx/acces 0.0s
=> CACHED [5/5] RUN ln -sf /dev/null /var/log/nginx/error 0.0s
=> exporting to image 0.1s
=> => exporting layers 0.0s
=> => writing image sha256:c7ad02f30caf44a1d264e90b21c757 0.0s
=> => naming to docker.io/library/ngx 0.0s
```

Listamos las imágenes para asegurarnos de que se ha creado

```
linux@linux-VirtualBox:~/ngnxx$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
a22bryanmar/ngnxx	latest	c7ad02f30caf	About an hour ago	192MB
a22bryanmar/ngnxx	v1	c7ad02f30caf	About an hour ago	192MB
ngnxx	latest	c7ad02f30caf	About an hour ago	192MB
hello-world	latest	74cc54e27dc4	4 weeks ago	10.1kB

Probamos localmente la imagen para ver que se inicia correctamente

```
linux@linux-VirtualBox:~/ngnxx$ sudo docker run -d -p 8080:80 ngnxx
ebd8f918e77d0a7919ed7a414cfc4144ff510f6dbcb9834dba05bdf8e797983b
linux@linux-VirtualBox:~/ngnxx$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ebd8f918e77d	ngnxx	"/docker-entrypoint..."	14 seconds ago	Up 13 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	youthfu

```
l_banach
```

Lanzamos el comando `docker login` para generar un token de inicio de sesión desde la consola

```
linux@linux-VirtualBox:~/ngnxx$ docker login
```

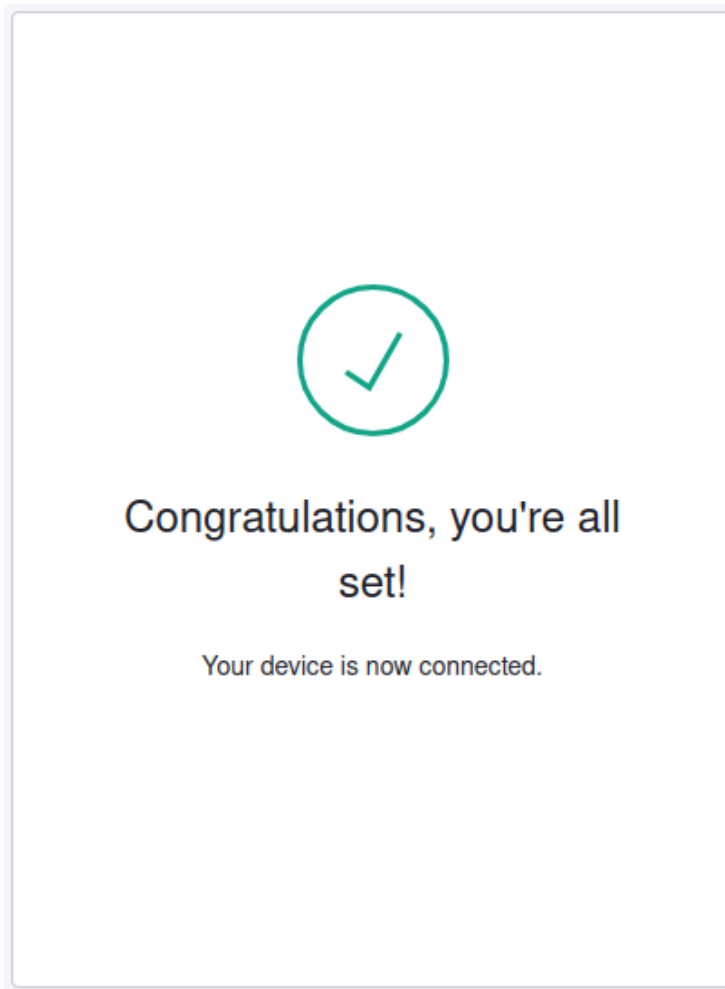
USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: **LDLQ-SBTF**
Press ENTER to open your browser or submit your device code here:
<https://login.docker.com/activate>

Waiting for authentication in the browser...

Si no hemos logueado correctamente, deberíamos recibir este mensaje en una ventana de nuestro navegador



Tageamos la imagen antes de subirla

```
linux@linux-VirtualBox:~/ngx$ sudo docker tag ngx ngx:v1
```

Subimos la imagen dockerhub

```
linux@linux-VirtualBox:~$ sudo docker push a22bryanmar/ngx:v1
The push refers to repository [docker.io/a22bryanmar/ngx]
a3d14b791b5d: Pushed
0b9a65755a5a: Pushed
94a5246fb54e: Pushed
12b573156051: Pushed
55e9644f21c3: Pushed
7d22e2347c12: Pushed
f6d5815f290e: Pushed
791f0a07985c: Pushed
cabea05c000e: Pushed
c68632c455ae: Pushed
5f1ee22ffb5e: Pushed
v1: digest: sha256:5484a94c54e79e3d5e551e6a0f2ec2c5e59aa36ac86b8f925b603f7374c5e55b size: 2606
```

Nos bajamos la imagen de dockerhub para cerciorarnos de que funciona todo

```
linux@linux-VirtualBox:~$ sudo docker pull a22bryanmar/nginx:v1
v1: Pulling from a22bryanmar/nginx
Digest: sha256:5484a94c54e79e3d5e551e6a0f2ec2c5e59aa36ac86b8f925b603f7374c5e55b
Status: Image is up to date for a22bryanmar/nginx:v1
docker.io/a22bryanmar/nginx:v1
```

Revisamos como tenemos el dockerfile subido correctamente a DockerHub, con su Warning.

a22bryanmar/nginx 

Last pushed 5 minutes ago · Repository size: 69.7 MB

WARNING! ----- VULNERABLE NGINX IMAGE WARNING! 

SECURITY 

Docker commands

To push a new tag to this repository:

```
docker push a22bryanmar/nginx:tagname
```

Public view

General Tags Image Management **BETA** Builds Collaborators Webhooks Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
 v1		Image	less than 1 day	7 minutes

[See all](#)

Automated builds

Manually pushing images to Docker Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.

Available with Pro, Team and Business subscriptions. [Read more about automated builds](#) 

Upgrade

Explicación: Ejemplo Complejo

1. ¿Cómo Funcionan los Dockerfiles?

Un Dockerfile es un archivo que contiene una serie de instrucciones que Docker utiliza para construir una imagen personalizada. En este caso, partimos de una imagen base confiable de Nginx, pero le aplicamos configuraciones y comandos que introducen múltiples vulnerabilidades de seguridad, convirtiéndola en una imagen peligrosa. El objetivo es demostrar cómo una imagen aparentemente legítima puede ser manipulada para comprometer sistemas.

El Dockerfile malicioso incluye prácticas como:

Ejecutar procesos como root, lo cual da permisos completos en el contenedor.

Definir variables de entorno con credenciales sensibles.

Exponer puertos innecesarios, como el 22 (SSH) y 3306 (MySQL), ampliando la superficie de ataque.

Instalar herramientas como netcat, curl o wget, que pueden facilitar movimientos laterales o extracción de datos.

Configurar Nginx para servir archivos del sistema, como /etc/passwd, lo cual expone información sensible.

Deshabilitar mecanismos de seguridad en Nginx, como límites de tamaño en peticiones, facilitando ataques tipo DoS o de inyección.

2. Generación del nginx.conf Vulnerable

Creamos un archivo de configuración de Nginx (nginx.conf) con parámetros inseguros, como:

Permitir acceso directo a rutas sensibles.

Eliminar restricciones de tamaño de cuerpo de las peticiones.

No aplicar restricciones IP ni headers de seguridad.

Este archivo es copiado en la imagen para que Nginx arranque con una configuración intencionalmente peligrosa.

3. Build de la Imagen Maliciosa

Utilizamos el comando docker build para generar la imagen con nuestro Dockerfile y el nginx.conf modificado.

Durante esta etapa, Docker empaqueta todo el entorno y genera una imagen lista para ser ejecutada o distribuida.

4. Validación de la Imagen

Listamos las imágenes disponibles con docker images para asegurarnos de que la construcción fue exitosa. Luego, probamos localmente con docker run para verificar que el contenedor se inicia y que Nginx sirve correctamente, incluso exponiendo los archivos sensibles, como se configuró.

5. Autenticación con Docker Hub

Usamos docker login desde consola para iniciar sesión en Docker Hub. Si el inicio de sesión falla, veremos un mensaje de error en la terminal o, en algunos casos, una advertencia desde el navegador que indica que no estamos autorizados a interactuar con el repositorio.

Este paso es clave para poder subir la imagen maliciosa al repositorio público.

6. Tag y Push de la Imagen

Etiquetamos la imagen con docker tag, asignándole un nombre que coincida con el repositorio de Docker Hub. Posteriormente, ejecutamos docker push para subir la imagen al repositorio.

Una vez en línea, cualquier usuario podría descargarla, sin notar a simple vista que contiene múltiples configuraciones inseguras.

7. Verificación desde Docker Hub

Para comprobar que todo se ha subido correctamente, descargamos la imagen nuevamente usando docker pull. Al ejecutarla localmente, verificamos que sigue funcionando con todas las vulnerabilidades activas.

Desde el portal de Docker Hub, inspeccionamos los detalles de la imagen, incluyendo el Dockerfile y cualquier advertencia generada por herramientas de escaneo automático. Esto demuestra cómo una imagen peligrosa puede ser pública y aparentemente legítima, si no se revisa adecuadamente.

Conclusión

Este ejercicio demuestra cómo una imagen aparentemente legítima (nginx) puede engañar a un usuario que cree estar descargando nginx. En el Proof Of Concept, la imagen no hace nada más que mostrar el mensaje “¡HAS SIDO HACKEADO!”, pero simula un ataque en cadena básico, ideal para nuestros fines demostrativos. Sería una versión muy sencilla de un ataque a la cadena de suministro, básicamente. En el Ejemplo Complejo, se demuestra cómo una imagen de Docker aparentemente legítima puede ser manipulada para incluir múltiples vulnerabilidades, como la exposición de credenciales, ejecución como root y configuración insegura de servicios como Nginx. Al construir, probar y subir esta imagen maliciosa, aprendemos cómo ocurren los ataques a la cadena de suministro y la importancia de revisar cuidadosamente las imágenes que usamos o distribuimos.

A través de estos ejemplos, hemos aprendido cómo los contenedores Docker pueden ser utilizados tanto para simplificar el despliegue de aplicaciones como para introducir riesgos de seguridad graves si no se manejan correctamente. Al construir imágenes maliciosas —ya sea con comandos peligrosos, configuraciones inseguras o contenido oculto— y distribuirlas a través de Docker Hub, comprendimos el funcionamiento de los ataques a la cadena de suministro y la importancia de aplicar buenas prácticas como no ejecutar procesos como root, evitar exponer credenciales, limitar los puertos abiertos, revisar los Dockerfiles y utilizar herramientas de escaneo de vulnerabilidades. Esta práctica nos ayuda a valorar la seguridad como parte esencial del ciclo de vida del software.

Apendice:

Link Repositorio Github: <https://github.com/dbrMy/P-ndola-4---Docker>

Documentacion Docker: <https://docs.docker.com/>

Link Repo Proof Of Concept DockerHub: <https://hub.docker.com/r/niko2005/ngnx>

Link Repo Ejemplo Complejo DockerHub: <https://hub.docker.com/r/a22bryanmar/ngnxx>